OS02 — Systèmes Distribués et Parallélisme

Optimisation par Colonie de Fourmis sur un Paysage Fractal

Vectorisation, OpenMP et MPI

Tianyi LIANG
Tianze XIA
Zhe Chen

Mars 2026

Table des matières

Introduction **3**

1 Environnement et mesure de référence **3**

1.1 Environnement de développement 3

1.2 Paramètres de la simulation 3

1.3 Protocole de mesure 3

1.4 Baseline séquentielle 4

2 Vectorisation — Restructuration AoS → SoA **5**

2.1 Principe 5

2.2 Résultats 5

2.3 Analyse du gain marginal 5

3 Parallélisation OpenMP (mémoire partagée) **6**

3.1 Stratégie 6

3.1.1 Avancement des fourmis (93.8%) 6

3.1.2 Évaporation des phéromones (6.2%) 6

3.2 Résultats 6

3.3 Analyse 7

3.3.1 Accélération maximale à 8 threads : 5.43× 7

3.3.2 Régression à 12 threads 8

3.3.3 Transfert de goulot d'étranglement 8

3.3.4 Comparaison avec la loi d'Amdahl 8

3.3.5 Reproductibilité 8

4 Parallélisation MPI — Méthode 1 (réplication de la carte) **9**

4.1 Conception 9

4.2 Volume de communication 9

4.3 Résultats 9

4.4 Analyse 9

5 Parallélisation MPI — Méthode 2 (partitionnement de la carte) **11**

5.1 Conception 11

5.1.1 Décomposition 1D par lignes 11

5.1.2 Schéma de communication par itération 11

5.1.3 Réduction du volume de communication 11

5.2 Résultats — MPI méthode 2 pur 12

5.3 Accélération super-linéaire des fourmis 13

5.4 Déséquilibre de charge 13
5.5 Comparaison méthode 1 vs méthode 2 14
5.6 Convergence algorithmique 14
5.7 Mode hybride MPI M2 + OpenMP 15

6 Synthèse et comparaison globale **16**
6.1 Classement des 4 approches 16
6.2 Analyse comparative 16

Conclusion **18**

A Guide de reproduction des expériences **19**
A.1 Prérequis 19
A.2 Compilation 19
A.3 Exécution des expériences 19
A.3.1 Baseline et SoA 19
A.3.2 OpenMP 19
A.3.3 MPI méthode 1 20
A.3.4 MPI méthode 2 20
A.4 Génération des graphiques 20
A.5 Liste des fichiers nécessaires 20

Introduction

Ce rapport présente l'optimisation progressive d'un simulateur de colonies de fourmis (ACO) sur un paysage fractal de 513×513 cellules. L'objectif est de mesurer et comparer les gains de performance obtenus par trois techniques de parallélisation : la restructuration mémoire (AoS→SoA), la parallélisation en mémoire partagée (OpenMP) et la parallélisation distribuée (MPI, deux méthodes).

Les expériences sont menées sur une machine virtuelle OrbStack Ubuntu 22.04 (aarch64, 12 cœurs ARM), avec GCC 11.4, OpenMPI et SDL2. Toutes les mesures sont effectuées en mode `headless` (sans rendu graphique) sur 5000 itérations, 3 exécutions par configuration, en retenant la médiane.

1 Environnement et mesure de référence

1.1 Environnement de développement

Composant	Détail
Système hôte	macOS (Apple Silicon)
Machine virtuelle	OrbStack — Ubuntu 22.04 (`ubuntujelly`)
Architecture	aarch64 (ARM64), 12 cœurs
Compilateur	GCC 11.4.0 (`-std=c++17 -O2 -march=native`)
MPI	OpenMPI (`mpic++ -fopenmp`)
SDL	SDL2 2.0.20

Table 1 – Configuration matérielle et logicielle

1.2 Paramètres de la simulation

Paramètre	Symbole	Valeur
Nombre de fourmis	m	5000
Grille	—	513×513
Coefficient d'exploration	ε	0.8
Paramètre de bruit	α	0.7
Coefficient d'évaporation	β	0.999
Position du nid	—	$(256, 256)$
Position de la nourriture	—	$(500, 500)$
Graine aléatoire	—	2026

Table 2 – Paramètres ACO

1.3 Protocole de mesure

Le code est instrumenté avec `std::chrono::high_resolution_clock` pour mesurer séparément : l'avancement des fourmis, l'évaporation ($\times\beta$) et la mise à jour des buffers. Le mode `--headless` désactive le rendu SDL (qui représente 86% du temps en mode graphique) pour mesurer uniquement le calcul.

1.4 Baseline séquentielle

Composant	Temps (ms)	ms/iter	Proportion
Avancement fourmis	8491.60	1.70	93.81%
Évaporation phéromones	556.60	0.11	6.15%
Mise à jour buffers	4.40	0.00	0.05%
Total boucle	**9051.20**	**1.81**	**100%**

Table 3 – Baseline séquentielle — médiane de 3 exécutions, 5000 itérations

La stabilité des mesures est excellente : sur les 3 exécutions, l'écart-type du temps total est de 9.30 ms (CV = 0.10%). Les résultats fonctionnels sont strictement identiques : première nourriture à l'itération 2942, total de 2440 unités collectées, confirmant le déterminisme du générateur pseudo-aléatoire (graine fixe 2026).

> L'avancement des fourmis domine à 93.81% du temps. Chaque fourmi effectue plusieurs déplacements par pas de temps (boucle `while(consumed_time < 1.)`) avec des accès mémoire dispersés dans la carte de phéromones (4.20 Mo), qui dépasse le cache L2 (∼256 Ko par cœur). Ce composant est la cible prioritaire de toute optimisation.

2 Vectorisation — Restructuration AoS → SoA

2.1 Principe

Dans la version originale (AoS), chaque fourmi est un objet `ant` contenant `{seed, state, position}`. Les 5000 objets sont stockés dans un `std::vector<ant>`. La restructuration SoA sépare les attributs en tableaux parallèles :

```
struct ants_soa {
    std::vector<int> positions_x, positions_y;
    std::vector<unsigned char> states;
    std::vector<std::size_t> seeds;
    std::size_t count;
};
```

Listing 1 – Structure SoA

Chaque tableau est parcouru séquentiellement, ce qui maximise l'utilisation des lignes de cache (16 `int`/ligne pour les positions, 8 `size_t`/ligne pour les graines).

2.2 Résultats

Composant	AoS (ms)	SoA (ms)	Diff.	Accél.
Avancement fourmis	8495.50	8441.50	−54.00	1.01×
Évaporation	567.10	555.80	−11.30	1.02×
Total boucle	**9067.00**	**9003.00**	−64.00	**1.01×**

Table 4 – Comparaison AoS vs SoA (médiane, 5000 itérations)

Les deux versions SoA et AoS montrent un coefficient de variation inférieur à 0.10%, confirmant la stabilité des mesures. La reproductibilité fonctionnelle est parfaite entre les deux versions (itération 2942, 2440 unités).

2.3 Analyse du gain marginal

Le gain de 0.71% est statistiquement réel (supérieur au bruit de mesure, CV $< 0.10\%$) mais pratiquement négligeable. L'explication tient au profil d'accès mémoire :

- Le goulot d'étranglement est l'accès aux **phéromones** (4.20 Mo, accès aléatoires), pas la disposition des fourmis (∼120 Ko en AoS, ∼100 Ko en SoA : les deux tiennent dans le cache L2). L'objet `ant` original est compact (∼24 octets) : une ligne de cache (64 octets) contient 2–3 fourmis, le gaspillage est limité.
- L'algorithme suit un schéma *scatter/gather* (chaque fourmi lit une position aléatoire dans les phéromones) et non un schéma *stream*, où la SoA excelle. La boucle interne a un nombre variable d'itérations par fourmi (dépendant de la valeur du terrain), empêchant la vectorisation SIMD par le compilateur.

> Ce résultat négatif est en soi informatif : il démontre que toute restructuration mémoire n'améliore pas automatiquement les performances. Le vrai levier est la **parallélisation**. Néanmoins, la restructuration SoA facilite le partitionnement des fourmis entre threads OpenMP et réduit le risque de *false sharing*.

3 Parallélisation OpenMP (mémoire partagée)

3.1 Stratégie

Deux régions parallèles sont introduites :

3.1.1 Avancement des fourmis (93.8%)

```
#pragma omp parallel for schedule(dynamic, 50) reduction(+:local_food)
for (size_t i = 0; i < ants.count; ++i) {
    // advance ant i ...
}
```

Listing 2 – Parallélisation de l'avancement des fourmis

L'ordonnancement dynamique est choisi car le nombre de pas par fourmi varie selon le terrain. Les écritures concurrentes dans le buffer de phéromones (`mark_pheronome`) sont tolérées sans protection atomique (méthode C) : la probabilité de collision est faible (∼4.70% par itération pour 5000 fourmis sur 513^2 cellules), et l'algorithme est intrinsèquement stochastique.

3.1.2 Évaporation des phéromones (6.2%)

```
#pragma omp parallel for collapse(2) schedule(static)
for (size_t i = 1; i <= dim; ++i)
    for (size_t j = 1; j <= dim; ++j)
        // evaporate cell (i,j)
```

Listing 3 – Évaporation parallèle

Boucle parfaitement parallèle ($513^2 \approx 263\,000$ itérations indépendantes).

3.2 Résultats

Threads	Fourmis	Évap.	M. à j.	Total (ms)	$S(p)$	$E(p)$	Nourr.
1	8367.60	735.00	4.40	9107.10	1.00×	100%	2440
2	4290.10	414.80	7.40	4720.50	1.93×	96.50%	2440
4	2236.70	266.60	9.60	2508.00	3.63×	90.80%	2440
8	1364.20	299.00	14.80	1678.20	5.43×	67.80%	2440
12	2190.10	1187.40	22.50	3400.80	2.68×	22.30%	2440

Table 5 – Passage à l'échelle OpenMP (médiane, 5000 itérations)

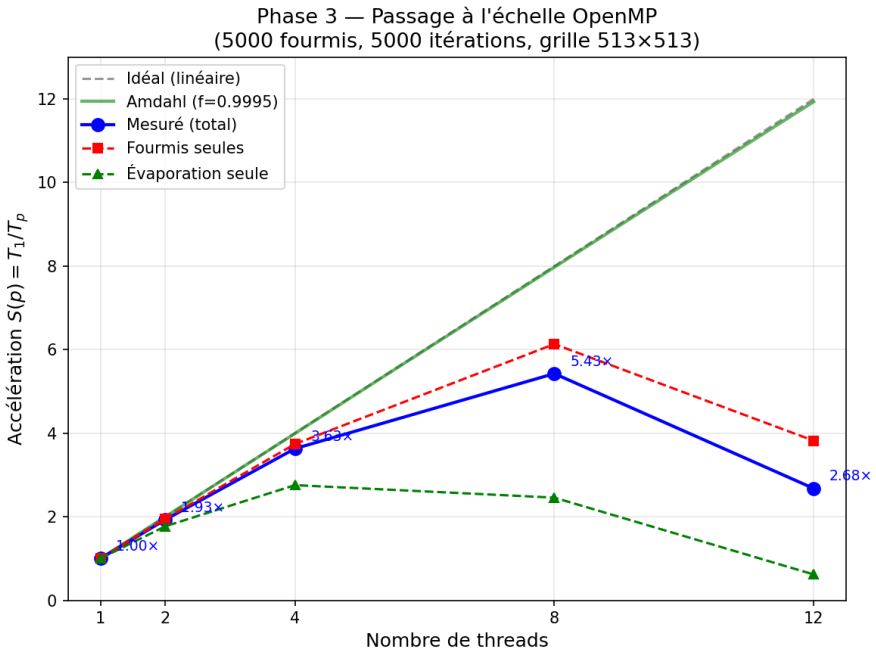


FIGURE 1 – Accélération OpenMP : mesurée vs loi d'Amdahl vs linéaire idéale

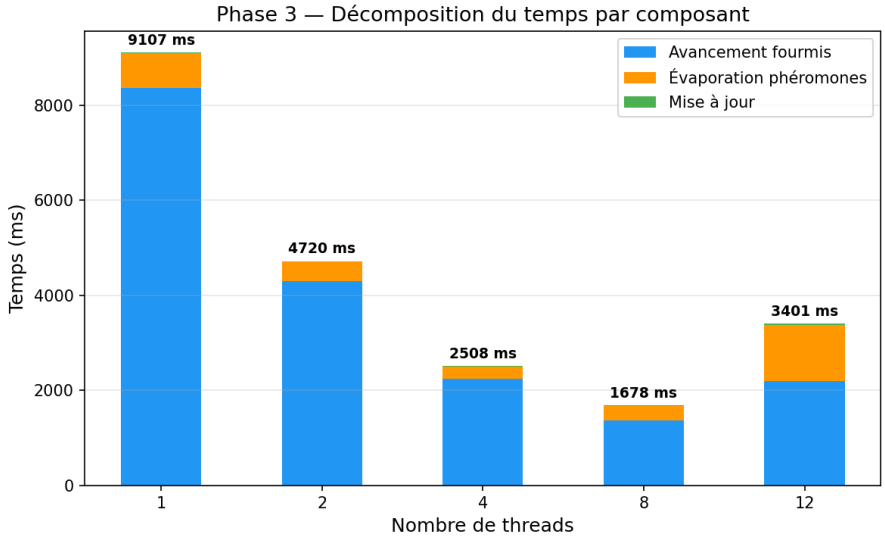


FIGURE 2 – Décomposition du temps par composant en fonction du nombre de threads

3.3 Analyse

3.3.1 Accélération maximale à 8 threads : 5.43×

Le meilleur résultat est obtenu avec 8 threads, réduisant le temps de 9107 ms à 1678 ms. L'avancement des fourmis atteint $S_{\text{fourmis}}(8) = 6.13\times$, confirmant l'efficacité du `schedule(dynamic)`.

La variabilité des mesures augmente avec le nombre de threads : le CV passe de 0.09% (1T) à 10.90% (8T), ce qui reflète la contention croissante pour les ressources partagées.

3.3.2 Régression à 12 threads

La performance chute à 2.68× pour 12 threads. Trois facteurs convergent :

1. **Saturation de la bande passante mémoire** : l'évaporation, kernel *memory-bound* (1 multiplication par lecture/écriture de 8 octets), passe de 299.00 ms (8T) à 1187.40 ms (12T) — soit 1.62× *plus lent qu'en séquentiel*.
2. **Contention du cache L2/L3** : la carte de phéromones (4.20 Mo) provoque du *thrashing* avec 12 threads accédant simultanément.
3. **Overhead d'ordonnancement** : $\lceil 5000/50 \rceil = 100$ tâches dynamiques par itération, avec barrières de synchronisation croissantes.

3.3.3 Transfert de goulot d'étranglement

La proportion de l'évaporation passe de 8.10% (1T) à 34.90% (12T). Résoudre le goulot des fourmis en révèle un autre sur l'évaporation.

3.3.4 Comparaison avec la loi d'Amdahl

Avec une fraction parallélisable $f = (8367.60 + 735.00)/9107.10 = 0.9995$, la loi d'Amdahl prédit $S(8) = 7.96\times$ et $S(12) = 11.9\times$, alors que les mesures donnent respectivement 5.43× et 2.68×. L'écart croissant démontre que le modèle d'Amdahl, qui ignore la contention mémoire et l'overhead de parallélisation, est trop optimiste pour ce type de charge *memory-bound*.

3.3.5 Reproductibilité

Les 15 exécutions (5 configs × 3 runs) produisent des résultats fonctionnels identiques (itération 2942, 2440 unités), confirmant que la tolérance aux conditions de course n'affecte pas la convergence. La probabilité de collision par itération reste faible : $P \approx m^2/(2N) \approx 4.7\%$ pour 5000 fourmis sur 513^2 cellules.

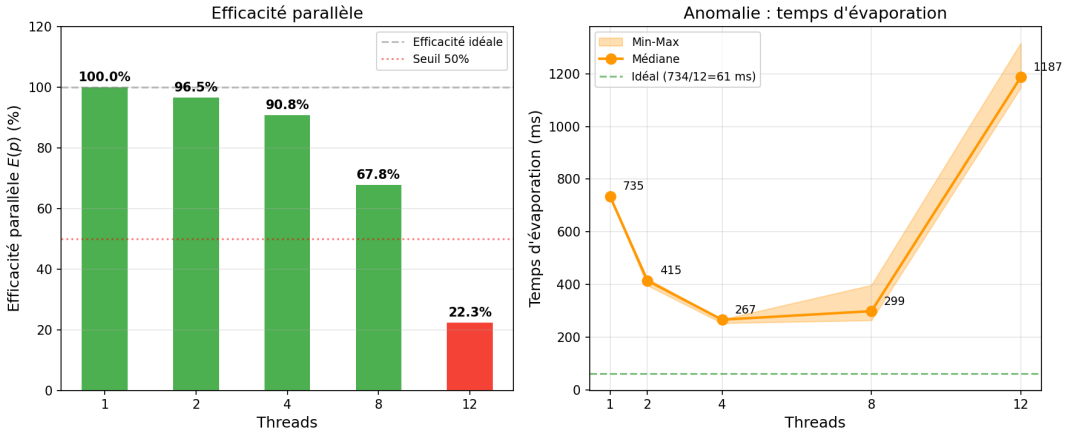


FIGURE 3 – Efficacité parallèle et anomalie du temps d'évaporation

4 Parallélisation MPI — Méthode 1 (réplication de la carte)

4.1 Conception

Chaque processus conserve une copie complète de la carte (513×513) et gère $\lfloor m/P \rfloor$ fourmis. La synchronisation suit le schéma :

1. Avancement des fourmis locales (N/P fourmis)
2. `MPI_Allreduce(MPI_MAX)` sur le buffer de phéromones (∼4.24 Mo)
3. Évaporation distribuée par blocs de lignes + `MPI_Allgatherv` (∼4.23 Mo)
4. `phen.update()` (échange de buffers)

La sémantique `MPI_MAX` est correcte car `mark_pheronome` ne fait qu'augmenter les valeurs (marquage = 1.0), conformément au cahier des charges.

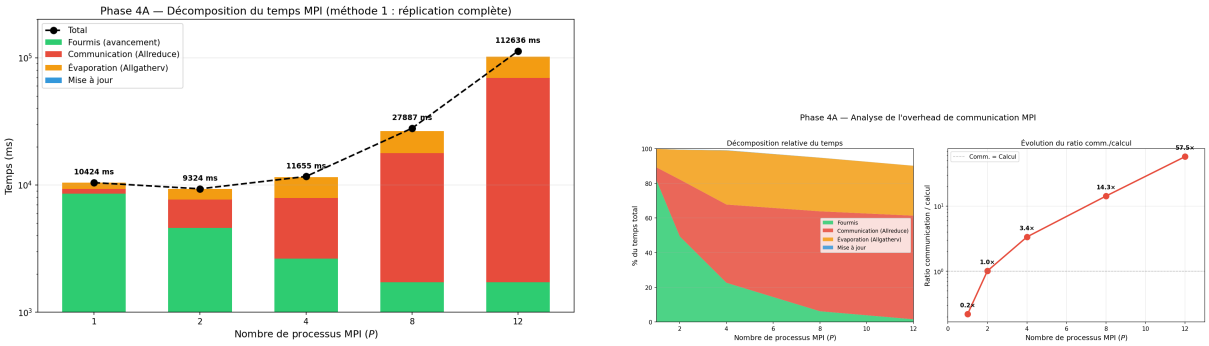
4.2 Volume de communication

Le volume total par itération est de **∼8.50 Mo** (4.24 Mo pour `Allreduce` + 4.23 Mo pour `Allgatherv`), **indépendant** du nombre de processus.

4.3 Résultats

P	Fourmis	Comm.	Évap.	Total (ms)	$S(P)$	$E(P)$	Nourr.
1	8540.10	764.70	1114.00	10423.50	1.00×	100%	2440
2	4607.40	3060.90	1602.10	9323.80	1.12×	55.90%	2651
4	2634.00	5266.10	3639.80	11655.30	0.89×	22.40%	2651
8	1717.80	16093.00	8628.60	27887.40	0.37×	4.70%	2651
12	1713.60	67392.70	32466.80	112636.50	0.09×	0.80%	2651

TABLE 6 – Passage à l'échelle MPI méthode 1 (médiane, 5000 itérations)



(a) Temps total et décomposition

(b) Ratio communication / calcul

FIGURE 4 – MPI méthode 1 — passage à l'échelle et ratio de communication

4.4 Analyse

La méthode 1 est **contre-productive** au-delà de 2 processus. L'accélération maximale n'est que de 1.12× ($P = 2$) et le programme est 11× plus lent à $P = 12$.

Le ratio communication/calcul explose : il passe de 0.08 ($P = 1$) à 57.55 ($P = 12$). Le volume de 8.50 Mo par itération est **constant** quel que soit P, car chaque processus synchronise la totalité de la carte. Ajouter des processus réduit le calcul mais augmente le coût des opérations collectives ($O(\log P)$ étapes avec volume total inchangé, plus contention sur le bus mémoire).

L'avancement des fourmis seul passe à l'échelle correctement ($S_{\text{fourmis}}(8) = 4.97\times$), ce qui confirme que le partitionnement du travail est efficace — mais ce gain est totalement absorbé par la communication.

Le mode hybride MPI+OpenMP a aussi été testé (cf. tableau 7) : la configuration $P = 1, T = 8$ produit 10434.50 ms, soit 14% plus lent que le pur OpenMP 8 threads (1678.20 ms). L'overhead provient de la couche MPI (`MPI_Allreduce` + `MPI_Allgatherv`) qui subsiste même avec un seul processus.

P	T	Cœurs	Total (ms)	S_{seq}	Nourr.
1	8	8	10434.50	0.87×	2440
2	4	8	9339.70	0.98×	2651
4	2	8	10884.60	0.84×	2651
2	2	4	8964.50	1.02×	2651
4	1	4	11090.40	0.82×	2651

Table 7 – Mode hybride MPI méthode 1 + OpenMP ($S_{\text{seq}} = 9107.10/T_{\text{hybrid}}$)

Conclusion : la parallélisation MPI avec réplication de la carte échoue pour cette configuration. Le ratio calcul/communication est trop défavorable : la grille 513×513 génère 8.50 Mo de données collectives par itération, indépendamment du nombre de processus. Ce résultat négatif est conforme à la prédiction du cahier des charges et illustre que la parallélisation distribuée n'est pas toujours bénéfique.

5 Parallélisation MPI — Méthode 2 (partitionnement de la carte)

5.1 Conception

5.1.1 Décomposition 1D par lignes

La grille est découpée en bandes horizontales. Chaque processus r possède $\lfloor 513/P \rfloor$ ou $\lceil 513/P \rceil$ lignes, avec une rangée de cellules fantômes (ghost cells) en haut et en bas.

```
   0                                  512    Y
 0  +-----------------------------------+
    |       Rank 0  (~128 lignes)       |
    |  ghost                            |
128 +-----------------------------------+
    |       Rank 1  (~129 lignes)       |
    |  ghost                            |
257 +-----------------------------------+
    |       Rank 2  (~128 lignes)       |
    |  ghost                            |
385 +-----------------------------------+
    | Rank 3  (~128 lignes)             |
    |                                   |
512 +-----------------------------------+

 X
```

Décomposition 1D pour $P = 4$ processus

5.1.2 Schéma de communication par itération

1. Avancement des fourmis locales, détection des dépassements de frontière
2. **Migration des fourmis** : protocole en 2 étapes via `MPI_Sendrecv` (échange des compteurs, puis des données)
3. **Halo exchange** : échange bidirectionnel des lignes de bordure (∼8 Ko par direction)
4. **Fusion MAX** des phéromones aux frontières
5. Évaporation locale + mise à jour des buffers

5.1.3 Réduction du volume de communication

Communication	**Méthode 1**	**Méthode 2**	**Réduction**
Phéromones (sync)	4.24 Mo (`Allreduce`)	∼32 Ko (halo)	×132
Évaporation	4.23 Mo (`Allgatherv`)	0 (locale)	∞
Migration fourmis	0	quelques Ko	nouveau
Total/iter	**∼8.50 Mo**	**∼50 Ko**	**×170**

Table 8 – Volume de communication : méthode 1 vs méthode 2

5.2 Résultats — MPI méthode 2 pur

P	Fourmis	Comm.	Évap.	M. à j.	Total (ms)	$S(P)$	$E(P)$	Nourr.
1	8354.70	2.00	475.00	5.50	8881.40	1.00×	100%	2440
2	3069.40	2345.40	222.40	74.30	5721.90	1.55×	77.60%	2092
4	1146.30	2123.80	101.20	61.90	3443.80	2.58×	64.50%	3609
8	371.20	1991.70	65.60	458.40	3216.50	2.76×	34.50%	3445
12	280.30	16774.90	77.20	6999.80	39814.60	0.22×	1.90%	3670

TABLE 9 – Passage à l'échelle MPI méthode 2 (médiane, 5000 itérations)

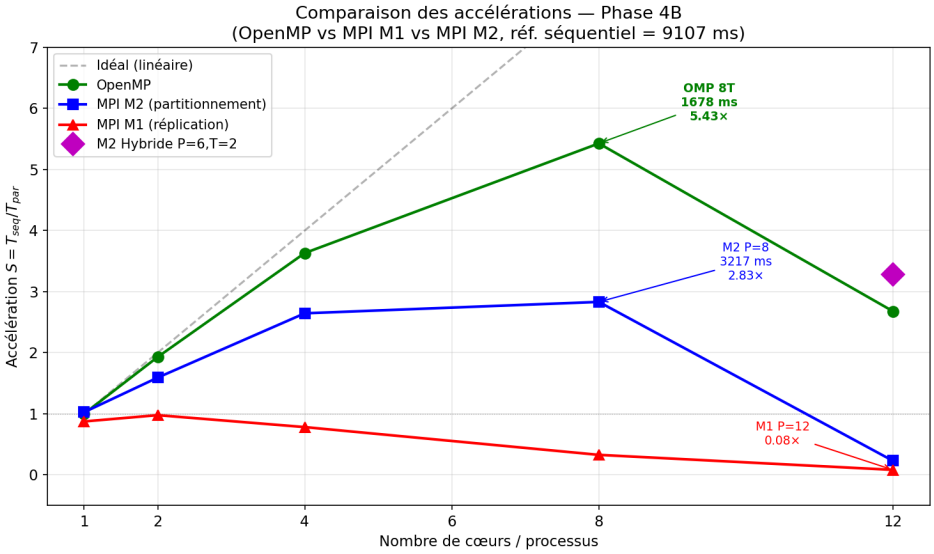


FIGURE 5 – Accélération comparée : OpenMP vs MPI M1 vs MPI M2

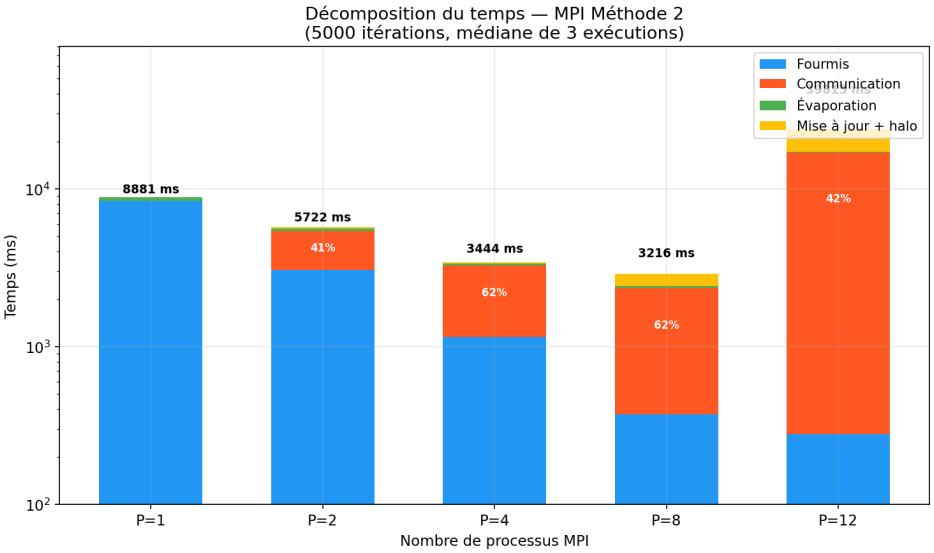


FIGURE 6 – Décomposition du temps — MPI méthode 2

5.3 Accélération super-linéaire des fourmis

Le résultat le plus remarquable concerne l'accélération du seul composant "fourmis" :

P	Fourmis (ms)	$S_{\text{fourmis}}(P)$	Carte locale
1	8354.70	1.00×	4.20 Mo
2	3069.40	2.72×	2.10 Mo
4	1146.30	7.29×	1.07 Mo
8	371.20	22.51×	0.55 Mo
12	280.30	29.81×	0.37 Mo

TABLE 10 – Accélération super-linéaire du composant "fourmis" — effet de cache

Pour $P = 8$, la carte locale (0.55 Mo) tient dans le cache L2 (∼256 Ko pour les accès fréquents), éliminant les *cache misses* qui dominent l'accès au tableau complet de 4.20 Mo. Combiné à la réduction du nombre de fourmis par processus, cela produit une efficacité de 281% — un avantage structurel de la méthode 2 que la méthode 1 ne peut pas atteindre.

5.4 Déséquilibre de charge

P	Distribution des fourmis	Ratio max/min
2	R0=1340, **R1=3660**	2.73×
4	R0=303, R1=1342, R2=1376, **R3=1979**	6.53×
8	R0=125, ..., **R7=1435**	11.48×
12	R0=33, ..., **R11=1349**	40.88×

TABLE 11 – Distribution finale des fourmis et ratio de déséquilibre

La source de nourriture $(500, 500)$ est toujours dans le dernier processus, qui accumule le plus de fourmis après convergence. Le ratio max/min atteint 40.88× pour $P = 12$, rendant le parallélisme inefficace : les processus rapides attendent le plus chargé à chaque barrière de synchronisation. Ce déséquilibre est le facteur limitant principal de la méthode 2.

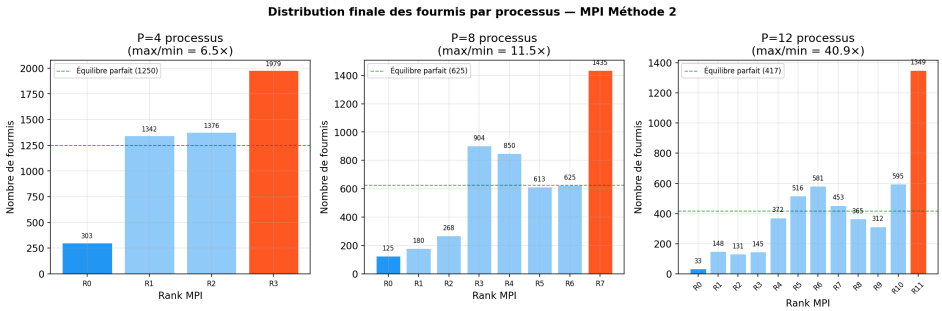


FIGURE 7 – Distribution des fourmis par processus

5.5 Comparaison méthode 1 vs méthode 2

P	M1 (ms)	M2 (ms)	Gain M2/M1	Réduction comm.
1	10423.50	8881.40	1.17×	×382
2	9323.80	5721.90	1.63×	×1.93
4	11655.30	3443.80	3.38×	×4.07
8	27887.40	3216.50	**8.67**×	×10.09
12	112636.50	39814.60	2.83×	×4.20

TABLE 12 – Comparaison directe méthode 1 vs méthode 2

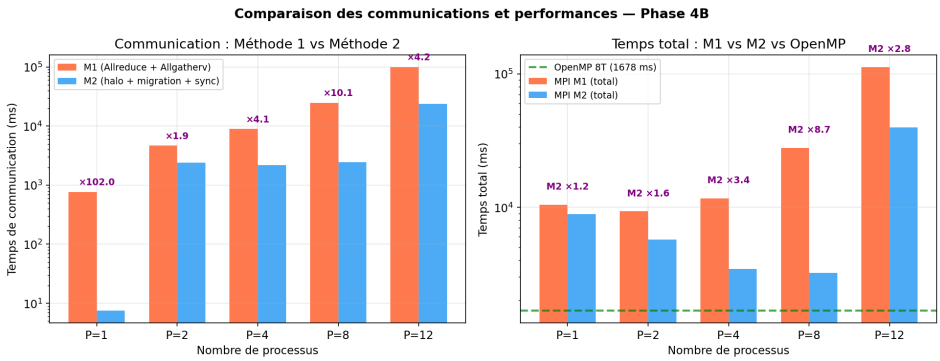


FIGURE 8 – Temps de communication : méthode 1 vs méthode 2

La méthode 2 est systématiquement plus rapide. À $P = 8$, elle est 8.67× plus rapide que la méthode 1, grâce à la combinaison de la communication réduite et de l'effet de cache.

5.6 Convergence algorithmique

La méthode 2 produit des résultats algorithmiques **différents** de la version séquentielle pour $P \geq 2$:

Config.	1ère nourriture (iter.)	Nourr. (5000 iter.)
Séquentiel / M2 $P = 1$	2942	2440
M2 $P = 2$	2944	2092
M2 $P = 4$	2672	3609
M2 $P = 8$	2583	3445
M2 $P = 12$	2553	3670

TABLE 13 – Convergence algorithmique — MPI méthode 2

Pour $P = 1$, M2 reproduit exactement les résultats séquentiels (validation de la correctitude). Pour $P \geq 4$, la première nourriture arrive plus tôt (2672 vs 2942) : le partitionnement de la carte modifie la diffusion des phéromones et crée des raccourcis locaux. Ces différences sont attendues : la méthode 2 est une approximation distribuée de l'algorithme séquentiel.

5.7 Mode hybride MPI M2 + OpenMP

P	T	Cœurs	Total (ms)	S_{seq}	Nourr.
1	8	8	8874.60	1.03×	2440
2	4	8	5553.60	1.64×	2092
4	2	8	3316.90	2.75×	3609
3	4	12	4153.00	2.19×	2136
4	3	12	3317.10	2.75×	3609
6	**2**	**12**	**2779.40**	**3.28×**	2103

TABLE 14 – Mode hybride MPI M2 + OpenMP ($S_{\text{seq}} = 9107.10/T_{\text{hybrid}}$)

L'ajout de threads OpenMP intra-processus apporte un gain limité. Pour $P = 4$, passer de $T = 1$ à $T = 2$ ne réduit le temps que de 3.70%, car le goulot est la synchronisation inter-processus (∼62% du temps), pas le calcul local. La meilleure configuration hybride est $P = 6, T = 2$ (2779.40 ms, 3.28×), qui bénéficie de bandes étroites (86 lignes, bonne localité cache) et d'une légère accélération OpenMP.

6 Synthèse et comparaison globale

6.1 Classement des 4 approches

Rang	Approche	Config.	Temps (ms)	S vs séq.	Cœurs
1	**OpenMP**	8 threads	**1678.20**	**5.43×**	8
2	MPI M2 hybride	P=6, T=2	2779.40	3.28×	12
3	MPI M2 pur	P=8	3216.50	2.83×	8
4	OpenMP	4 threads	2508.00	3.63×	4
5	MPI M2 hybride	P=4, T=2	3316.90	2.75×	8
6	MPI M1	P=2	9323.80	0.98×	2
7	MPI M1	P=12	112636.50	0.08×	12

TABLE 15 – Classement des approches (référence séquentielle SoA = 9107.10 ms)

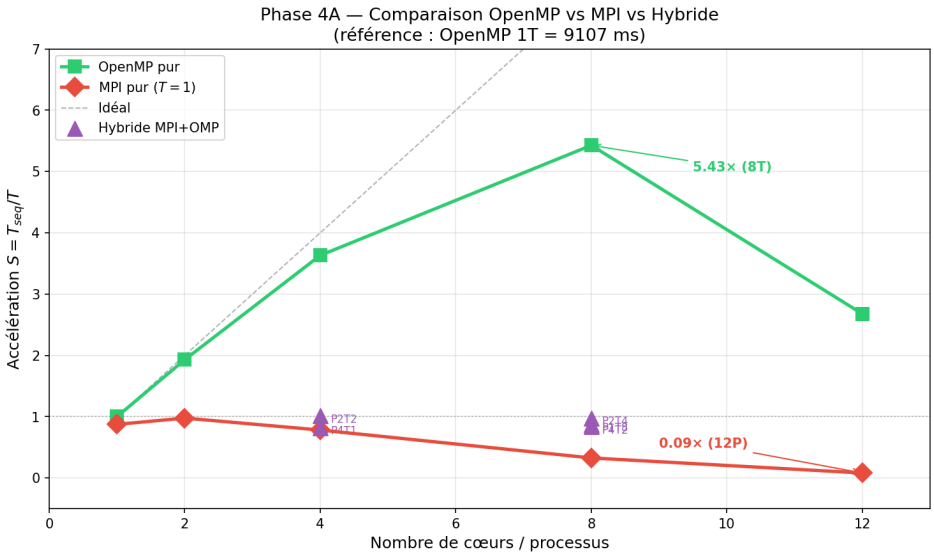


FIGURE 9 – Comparaison des accélérations : OpenMP vs MPI M1 vs MPI M2 hybride

6.2 Analyse comparative

	OpenMP	MPI M1	MPI M2
Communication/iter	0	∼8.50 Mo	∼50 Ko
Équilibre de charge	Dynamique (bon)	Statique (bon)	Statique (mauvais)
Effet de cache	Aucun (4.20 Mo)	Aucun (4.20 Mo)	Fort (0.55 Mo à $P = 8$)
Facteur limitant	Bande passante mém.	Volume comm.	Déséquilibre charge
Extensibilité multi-nœud	Non	Oui	Oui
Meilleure accél.	5.43× (8T)	1.12× (2P)	3.28× (6P×2T)

TABLE 16 – Synthèse qualitative des trois approches

Pourquoi OpenMP reste supérieur sur un nœud : aucune communication, aucune sérialisation de données, pas de migration de fourmis, ordonnancement dynamique avec équilibre de

charge automatique. Le seul inconvénient est l'impossibilité de dépasser une machine physique.

Pourquoi M2 bat M1 : la réduction de communication ($\times 170$) et l'effet de cache (carte locale dans L2) compensent largement le déséquilibre de charge. La méthode 2 est la seule approche MPI viable pour cette application.

Le compromis fondamental : la méthode 2 gagne sur le volume de communication mais perd sur l'équilibre de charge. La méthode 1 a un équilibre parfait mais un volume de communication prohibitif. Ni l'une ni l'autre ne bat OpenMP sur un seul nœud, mais seul MPI peut s'étendre sur un cluster multi-nœuds. Pour une telle application distribuée à plus grande échelle, la méthode 2 avec rééquilibrage dynamique des fourmis serait la voie à suivre.

Conclusion

Ce projet a permis d'explorer quatre niveaux d'optimisation d'un simulateur ACO :

1. **Vectorisation (SoA)** : gain négligeable (+0.71%), car le goulot est l'accès aléatoire aux phéromones, pas la disposition des fourmis.
2. **OpenMP** : accélération de 5.43× avec 8 threads, limitée par la bande passante mémoire. Régression à 12 threads due à la saturation du bus mémoire lors de l'évaporation.
3. **MPI méthode 1** : échec complet (accélération maximale 1.12×), dominé par 8.50 Mo/itération de communication collective.
4. **MPI méthode 2** : amélioration considérable (3.28× en hybride), grâce à la communication réduite (∼50 Ko/itération) et à l'effet de cache super-linéaire. Limitée par le déséquilibre de charge (ratio 40.88× à $P = 12$).

Le résultat principal est que, sur un seul nœud, la parallélisation en **mémoire partagée** (OpenMP) surpasse systématiquement la parallélisation **distribuée** (MPI), car l'overhead de communication et de synchronisation domine dès que le ratio calcul/communication est défavorable. La méthode 2 démontre néanmoins l'intérêt du partitionnement de données pour les applications à empreinte mémoire importante : l'effet de cache (accélération super-linéaire des fourmis, 22.51× à $P = 8$) constitue un avantage structurel qui pourrait être décisif sur un cluster multi-nœuds avec des grilles de plus grande taille.

A Guide de reproduction des expériences

A.1 Prérequis

Les expériences ont été réalisées sur une VM OrbStack Ubuntu 22.04 (aarch64, 12 cœurs ARM). Les dépendances nécessaires sont :

```
sudo apt update
sudo apt install -y build-essential g++ libsdl2-dev \
    openmpi-bin libopenmpi-dev
```

Listing 4 – Installation des dépendances

Vérifier les versions :

- `g++ -version` → GCC 11.4.0 ou supérieur
- `mpic++ -version` → OpenMPI wrapping GCC
- `sdl2-config -version` → SDL2 2.0.x

A.2 Compilation

Le fichier `Make_linux.inc` contient la configuration du compilateur :

```
cd projet/src
make clean && make all
```

Listing 5 – Compilation de toutes les cibles

Ceci génère les exécutables suivants :

- `ant_simu.exe` — version séquentielle originale (AoS)
- `ant_simu_soa.exe` — version SoA
- `ant_simu_omp.exe` — version OpenMP (SoA)
- `ant_simu_mpi.exe` — MPI méthode 1
- `ant_simu_mpi2.exe` — MPI méthode 2 (partitionnement)

A.3 Exécution des expériences

A.3.1 Baseline et SoA

```
# Baseline (3 runs)
for i in 1 2 3; do
    ./ant_simu.exe --headless -n 5000
done 2>&1 | tee ../results/baseline_3runs.txt

# SoA (3 runs)
for i in 1 2 3; do
    ./ant_simu_soa.exe --headless -n 5000
done 2>&1 | tee ../results/soa_3runs.txt
```

Listing 6 – Mesure de la baseline et SoA

A.3.2 OpenMP

```
for T in 1 2 4 8 12; do
    for i in 1 2 3; do
        ./ant_simu_omp.exe --headless -n 5000 -t $T
```

```
    done
done 2>&1 | tee ../results/omp_scaling.txt
```

Listing 7 – Passage à l'échelle OpenMP

A.3.3 MPI méthode 1

```
for P in 1 2 4 8 12; do
    for i in 1 2 3; do
        mpirun -np $P --oversubscribe \
            ./ant_simu_mpi.exe --headless -n 5000 -t 1
    done
done 2>&1 | tee ../results/mpi_scaling.txt
```

Listing 8 – Passage à l'échelle MPI méthode 1

A.3.4 MPI méthode 2

```
# MPI M2 pur
for P in 1 2 4 8 12; do
    for i in 1 2 3; do
        mpirun -np $P --oversubscribe \
            ./ant_simu_mpi2.exe --headless -n 5000 -t 1
    done
done 2>&1 | tee ../results/mpi2_scaling.txt

# MPI M2 hybride
for P in 1 2 3 4 6; do
    for T in 2 4; do
        for i in 1 2 3; do
            mpirun -np $P --oversubscribe \
                ./ant_simu_mpi2.exe --headless -n 5000 -t $T
        done
    done
done 2>&1 | tee ../results/mpi2_omp_hybrid.txt
```

Listing 9 – Passage à l'échelle MPI méthode 2

A.4 Génération des graphiques

Trois scripts Python (nécessitant `matplotlib` et `numpy`) sont fournis dans `results/` :

```
cd projet/results
python3 plot_omp_scaling.py     # -> omp_*.png
python3 plot_mpi_scaling.py     # -> mpi_*.png
python3 plot_mpi2_results.py    # -> mpi2_*.png
```

Listing 10 – Génération des figures

A.5 Liste des fichiers nécessaires

Pour la reproductibilité complète, les fichiers suivants doivent être inclus :

Catégorie	Fichier	Rôle
Sources C++	`src/ant_simu.cpp`	Baseline séquentielle
	`src/ant_simu_soa.cpp`	Version SoA
	`src/ant_simu_omp.cpp`	Version OpenMP
	`src/ant_simu_mpi.cpp`	MPI méthode 1
	`src/ant_simu_mpi2.cpp`	MPI méthode 2
Headers	`src/ant.hpp`	Classe ant (AoS)
	`src/ant.cpp`	Implémentation ant
	`src/ant_soa.hpp`	Structure SoA
	`src/ant_soa_omp.hpp`	SoA + OpenMP
	`src/pheronome.hpp`	Carte de phéromones
	`src/fractal_land.hpp/cpp`	Terrain fractal
	`src/renderer.hpp/cpp`	Rendu SDL2
	`src/window.hpp/cpp`	Fenêtre SDL2
	`src/basic_types.hpp`	Types de base
	`src/rand_generator.hpp`	Générateur aléatoire
Build	`src/Makefile`	Fichier de compilation
	`src/Make_linux.inc`	Config. Linux
Scripts	`results/plot_omp_scaling.py`	Graphiques OpenMP
	`results/plot_mpi_scaling.py`	Graphiques MPI M1
	`results/plot_mpi2_results.py`	Graphiques MPI M2
Données	`results/*.txt`	Données brutes
	`results/*.png`	Figures générées
Rapport	`rapport.tex`	Ce document

Table 17 – Fichiers nécessaires à la soumission

Références

[1] Supports de cours OS02, ENSTA Paris, 2026.

[2] M. Dorigo et T. Stützle, *Ant Colony Optimization*, MIT Press, 2004.

[3] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities”, *AFIPS Conference Proceedings*, vol. 30, pp. 483–485, 1967.